

EXPERIMENT-5

Student Details

Name: R.Varshitha

Roll Number: 24501a4450

Dataset: Student Performance Dataset

1.Business Understanding

1.1 Problem Statement & Relevance

Student academic performance can be influenced by age, gender, study hours, attendance and test scores. Predicting whether a student will pass or fail can help identify academic outcomes.

Problem Statement: Develop a machine learning classification model that predicts whether a student will pass or fail using the available student performance data.

Relevance: The problem can support identification of student performance outcomes and academic decision making.

1.2 Expected Benefits

The predictions can help identify students who are likely to pass or fail. This can help teachers and educational institutions monitor student performance and provide appropriate academic support.

2.Data Understanding

2.1Dataset Structure

Dataset Name: Student Performance Dataset

Variables: Student_ID, Age, Gender, Study_Hours, Attendance(%), Test_Score and Grade.

The program also creates Study_Efficiency and Pass_Fail..

Target: Pass_Fail

2.2 Data Dictionary

Column	Description	Type	Role
Student_ID	Student identification number	Identifier	Removed
Age	Age of the student	Numerical	Input feature
Gender	Gender of the student	Categorical	Input feature

Column	Description	Type	Role
Study_Hours	Student study hours	Numerical	Input feature
Attendance(%)	Student attendance percentage	Numerical	Input feature
Test_Score	Student test score	Numerical	Input feature
Grade	Student academic grade	Categorical	Used to create target
Study_Efficiency	Study hours divided by Attendance(%)	Numerical	Engineered feature
Pass_Fail	Student pass/fail classification	Binary	Target variable

2.3 Data Quality Observations

Missing values are checked using `df.isnull().sum()`.

Missing numerical values are filled with the median of each numerical column.

Duplicate records are checked using `df.duplicated().sum()`.

Student_ID is dropped because it is an unnecessary identifier.

Gender is label encoded.

Grade is one-hot encoded.

Outliers are detected using the IQR method and rows containing numerical outliers are removed.

Code Required: Printing summary statistics.

```
df.info()
```

```
df.head()
```

```
df.shape
```

```
df.describe()
```

```
print("number of duplicate records:", df.duplicated().sum())
```

```
df.isnull().sum()
```

```
print(df.columns)
```

3 Data Preparation

3.1 Cleaning Steps

The dataset is cleaned by checking missing and duplicate records, filling missing numerical values with medians, removing Student_ID, encoding categorical variables, detecting outliers using IQR and removing rows containing outliers.

3.2 Feature Engineering

Study_Efficiency is created as Study_Hours divided by Attendance(%). Gender is label encoded and Grade is one-hot encoded. Pass_Fail is created from Grade_F, with 0 for Grade F and 1 for all other grades.

Source Code:

```
df['Study_Efficiency'] = df['Study_Hours'] / df['Attendance(%)']
```

```
df['Pass_Fail'] = np.where(df['Grade_F'] == True, 0, 1)
```

3.3 Train/Test Split

The final features X and target y are separated and the data is split into 80% training data and 20% testing data using random_state=42. Training data is used to train the model and testing data is used to evaluate unseen data.

```
X = df.drop('Pass_Fail', axis=1)
```

```
y = df['Pass_Fail']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Coding Required:

```
numeric_columns = df.select_dtypes(include=np.number).columns
```

```
for column in numeric_columns:
```

```
    df[column].fillna(df[column].median(), inplace=True)
```

```
df.drop('Student_ID', axis=1, inplace=True)
```

```
le = LabelEncoder()
```

```
df['Gender'] = le.fit_transform(df['Gender'])
```

```
df = pd.get_dummies(df, columns=['Grade'])
```

```
df['Study_Efficiency'] = df['Study_Hours'] / df['Attendance(%)']
```

```

numeric_df = df.select_dtypes(include=np.number)

Q1 = numeric_df.quantile(0.25)

Q3 = numeric_df.quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - (1.5 * IQR)

upper_limit = Q3 + (1.5 * IQR)

outliers = ((numeric_df < lower_limit) | (numeric_df > upper_limit)).sum()

df = df[~((df.select_dtypes(include=np.number) < lower_limit) | (df.select_dtypes(include=np.number) >
upper_limit)).any(axis=1)]

df['Pass_Fail'] = np.where(df['Grade_F'] == True, 0, 1)

grade_columns = ['Grade_A', 'Grade_A+', 'Grade_B', 'Grade_C', 'Grade_F']

df.drop(columns=grade_columns, inplace=True)

X = df.drop('Pass_Fail', axis=1)

y = df['Pass_Fail']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

4 Modelling

4.1 Model Approach

Algorithm Used: Logistic Regression. It suits this problem because the target variable Pass_Fail has two classes, Pass and Fail. The model learns the relationship between the prepared student features and this binary target.

4.2 Model Training

The selected numerical features are standardized using StandardScaler. A Logistic Regression model is then created, fitted using the training data and used to predict the test data.

Source code:

```

scaler = StandardScaler()

cols_to_scale = ['Age', 'Study_Hours', 'Attendance(%)', 'Test_Score', 'Study_Efficiency']

X_train[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])

```

```
X_test[cols_to_scale] = scaler.transform(X_test[cols_to_scale])
```

4.3 Model Outputs & Interpretation

The Logistic Regression model generates Pass/Fail predictions for the test data. The predictions are stored in `y_pred`. The program also calculates training and testing accuracy.

Code Required:

```
scaler = StandardScaler()

cols_to_scale = ['Age', 'Study_Hours', 'Attendance(%)', 'Test_Score', 'Study_Efficiency']

X_train[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])

X_test[cols_to_scale] = scaler.transform(X_test[cols_to_scale])

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(y_pred)
```

5 Evaluation

5.1 Appropriate Metric Selection

Accuracy measures the proportion of correctly classified Pass/Fail records. A confusion matrix shows correct and incorrect classifications. The ROC curve shows the True Positive Rate against the False Positive Rate, and the AUC score measures the area under the ROC curve.

5.2 Interpretation

Training and testing accuracy are calculated using `model.score()`. The confusion matrix shows the classification results for the test data. The AUC score is calculated from the ROC curve. The exact numerical results are not included in the supplied Python source, so the executed output pictures can be pasted.

Code Required:.

```
train_accuracy = model.score(X_train, y_train)

test_accuracy = model.score(X_test, y_test)

print("Training Accuracy:", train_accuracy)

print("Testing Accuracy:", test_accuracy)

cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:")

print(cm)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel('Predicted Labels')

plt.ylabel('True Labels')

plt.title('Confusion Matrix')

plt.show()

fpr, tpr, thresholds = roc_curve(y_test, y_pred)

auc_score = roc_auc_score(y_test, y_pred)

print("AUC Score:", auc_score)

plt.plot(fpr, tpr, label='AUC={:.2f}'.format(auc_score))

plt.plot([0, 1], [0, 1], 'k--')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')
```

6 Conclusion

The Student Performance dataset is processed and a Logistic Regression model is trained to classify students into Pass and Fail.

The code performs missing-value handling, categorical encoding, Study_Efficiency feature engineering, IQR-based outlier removal, scaling and Logistic Regression classification.

Pass_Fail is created from Grade_F, with 0 representing Fail and 1 representing Pass.

The supplied Python source does not contain the executed numerical results. Future work can compare other classification algorithms and additional classification metrics.

7. Appendix

Confusion Matrix

Source Code:

```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")

print(cm)
```

```

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

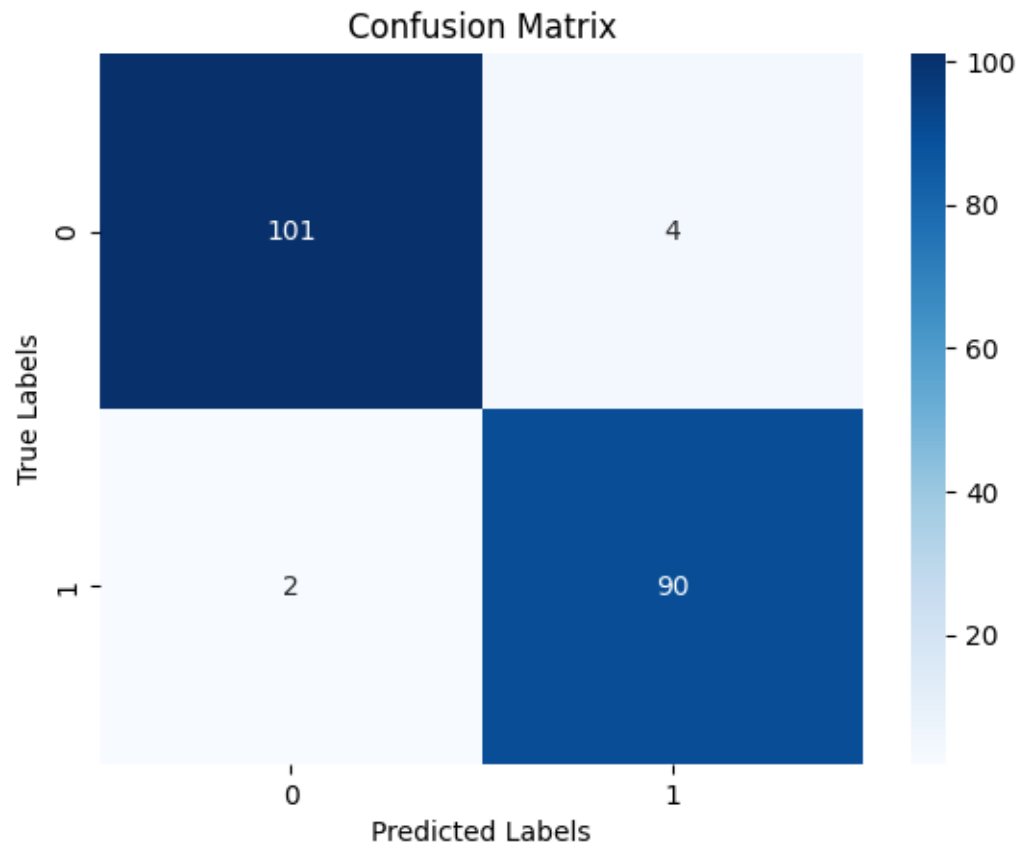
plt.xlabel('Predicted Labels')

plt.ylabel('True Labels')

plt.title('Confusion Matrix')

plt.show()

```



- **ROC Curve and AUC Score**

Source Code:

```

fpr, tpr, thresholds = roc_curve(y_test, y_pred)
auc_score = roc_auc_score(y_test, y_pred)
print("AUC Score:", auc_score)
plt.plot(fpr, tpr, label='AUC={:.2f}'.format(auc_score))
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.show()

```

